Informe Desafio 2
Informatica 2

Alejandro Escobar Arbelaez

Sergio Luis Laverde Florez

Universidad De Antioquia

Decisiones arquitectonicas importantes:

1. Se decidio solo organizar los arreglos principales que guardan los usuarios y los chats del sistema.

2. Se dedicio, en las clases que lo requieran, no guardar como identificador de usuario dueño su username si no un puntero hacia el usuario.

3. Se decidio que los chats se van a ordenar y redicir en un arreglo maestro paralelo de Chats.

4. Se decidio manejar la interfaz y logica general del programa por medio de namespaces en vez de clases controladoras.

5. Para el ordenamiento del arreglo de chats, se utilizara un identificador que se tratara de la concatenacion de los usrname de los participantes con un caracter especial intermedio, al igual que para la busqueda.

Esto permite la utilizacion de algoritmos como Binary Search.

6. No se guardo en memoria informacion referente a los chats, ya que esto es informacion determinista que se puede construir de manera barata.

Resumen clases:

1. Usr:
Abreviatura de user, esta clase guarda y trabaja todo con respecto a los usuarios de la plataforma.
Esta clase tiene relacion con todas las otras clases del siste, Al poseer en si atributos de la clase Chat y Album

2. Chat:
Clase que agrupa y permite la visualizacion de los mensajes, un chat entre dos usuarios se genera al primer mensjae entre los dos.

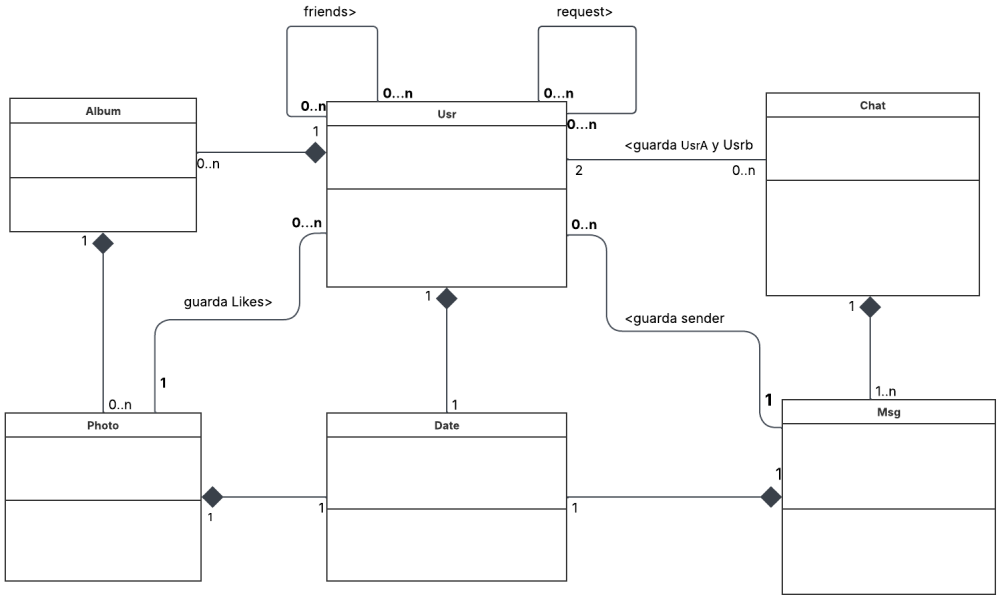
3. Album:
clase que sirve de coleccion de photos, aportando tambien desde sus atributos la configuracion de privacidad de las fotos.

4. Msg:
Clase especializada en el guardado estructurado de los atributos de los mensajes entre usuarios

5. Photo
Clase que guarda en si toda la informacion de una foto perteneciente a un album

6. Date:
Clase fecha, contiene miembros solo referentes a la operacion de datos de su propia clase y tipos de datos triviales.

DIAGRAMA DE CLASES RESUMEN:



Resumen Namespaces:

1. Validators:

Namespace sencillo encargado unicamente de validaciones de frontera.

(ya que esta version no permite la creacion de usuarios nuevos, no se requirio llenar mucho este modulo, pero se dejo estructurado)

2. dataManager:

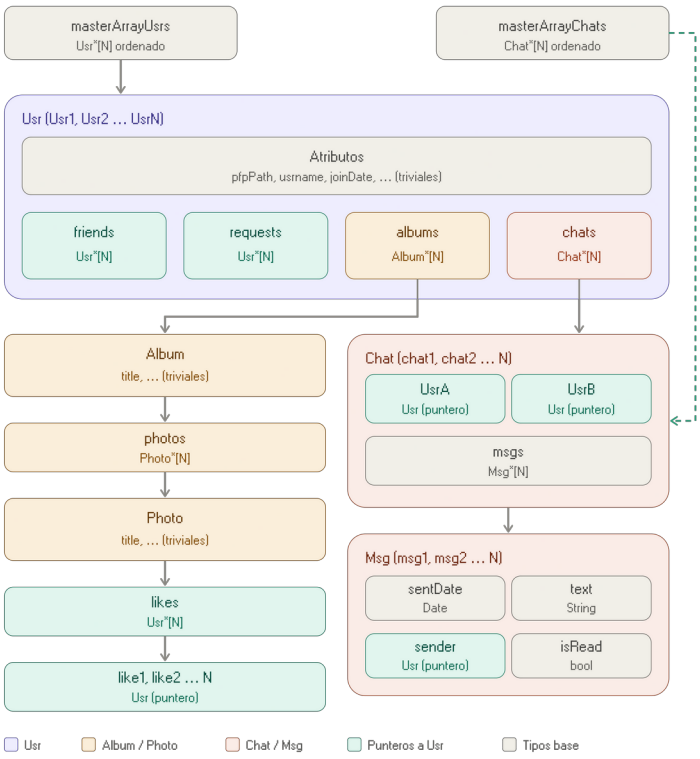
Encargado de cargar la estructura de datos, guardar los datos en archivos de texto plano y liberar la memoria del programa.

3. Core: Encargado de la logica principal del programa, ahi reciden las funciones de busqueda en la estructura de datos

4. UdeaBook: principalmente encargado de la interfaz y la interaccion con el usuario

la estructura de datos planteada para el almacenamiento de datos se ve representada en este diagrama en texto plano:

< DataStructure (UDEABOOK)>



Formato de los archivos utilizados para el almacenamiento de los datos:

1. Users.csv:

usrname1;email;pwd;city;country;dob;joinDate;friend1,friend2,friend3,friend4;request1
usrname2;email;pwd;city;country;dob;joinDate;friend1,friend2,friend3;request1,request2,request3
usrname3;email;pwd;city;country;dob;joinDate;friend1;request1,request2
usrname4;email;pwd;city;country;dob;joinDate;friend1,friend2,friend3,friend4;request1,request2

2. Chats.txt:

#usrname1;usrname2
usrname1;sentDate;isRead;text
usrname1;sentDate;isRead;text
usrname2;sentDate;isRead;text
#usrname3;usrname4
usrname3;sentDate;isRead;text

3. Albums.txt:

#usrname1
@albumtitle1;visibility
phototitle1;date;fileSize;likeUsrname1,likeUsrname2,likeUsrname3
phototitle2;date;fileSize;likeUsrname1,likeUsrname2
@albumtitle2;visibility
phototitle1;date;fileSize;likeUsrname1,likeUsrname2,likeUsrname3
#usrname2
@albumtitle1;visibility
phototitle1;date;fileSize;likeUsrname1,likeUsrname2,likeUsrname3
phototitle2;date;fileSize;likeUsrname1,likeUsrname2

Algoritmo principal:

1. Fase de inicializacion:

Se inicializan los punteros de los arreglos maestros y sus variables de control

2. Carga y construccion de la estructura de datos:

primero se leera users.csv y se llenara el arreglo maestro de users con los atributos triviales de cada usuario.

Segundo, se volvera a leer el archivo users.csv por segunda vez, pues ya una vez teniendo a todos los usuarios conocidos, podemos llenar los arreglos de friends y de requests que son atributos del tipo Usr* de la clase Usr

tercero, se lee el archivo albums.txt y se llena el arreglo de albums en cada usuario

y por ultimo, se lee el archivo chats.txt y se carga todo en el arreglo maestro de chats y a la vez, se va completando el ultimo dato faltante de los usuarios: el arreglo de chats que tienen abiertos

3. Se registra cuantas iteraciones y cuanta memoria se ha utilizado en el programa hasta ese punto

4. se entra al panel principal de opciones para el usuario, a partir de ahi debera de escoger si entrar al perfil de un amigo, ver sugerencias, entrar a un chat o configurar sus amistades

5. guardado y destruccion de la estructura de datos:

despues de que el usuario decida salir del programa, se guardan todos los datos en sus respectivos archivos para despues destruir la estructura de datos completa

Problemas que se afrontaron en el desarrollo:

durante el desarrollo de este desafío se afrontaron problemas principalmente en la coordinación entre los usuarios debido a que era la primera vez que ambos participantes trabajaban con ramas además de que uno venia de trabajar solo en el primer desafío, abundaron los conflictos en los merge y hubo diferencias en tanto a como desarrollar una idea para el codigo, sin embargo gracias al dialogo y la buena comunicación se lograron pasar todos estos problemas para llegar a una solución satisfactoria.

Evolución de la solución:

Se empezó por crear la clase fecha primero pues era usada por las demás clases, después se separo el desarrollo por dos caminos donde cada integrante del grupo tomo uno, el primero eran las clases photo y álbum y el segundo eran las clases Msg(mensaje) y chat. Al final se hizo la clase usr(usuario), después se corrigieron muchas cosas de la unión de estas clases e incluso se altero la estructura del diagrama de clases planteado para una mejor solución, después de tener las clases se continuo con hacer la estructura del programa que controlaba los archivos y el guardado alternamente por otro integrante se hizo la estructura que manejaba la interfaz y toda la interacción con el usuario además procesos internos del core del programa. Al final se fusionaron los trabajos, se corrigieron errores y se llego a la solución.